EVALUASI TENGAH SEMESTER (ETS)

Algoritma dan Pemrograman

SISTEM MONITORING LEVEL AIR TANDON GEDUNG BERBASIS RUST

Disusun oleh:

Nama : Tiara Hemas Eddy Putri
NRP : 2042251019

Nama : Craffty Hana Marshallya Chandra
NRP : 2042251054

Dosen Pengampu:

Ahmad Radhy, S.Si., M.Si.

Departemen Teknik Instrumentasi
Institut Teknologi Sepuluh Nopember

Daftar Isi

1 Pendahuluan **2**
1.1 Latar Belakang . . . 2
1.2 Tujuan . . . 2

2 Studi Kasus Instrumentasi **2**
2.1 Deskripsi Sistem . . . 2
2.2 Parameter Sistem . . . 3
2.3 Status Level Air . . . 3
2.4 Aturan Kontrol Pompa . . . 3

3 Algoritma dan Flowchart **3**
3.1 Algoritma Sistem . . . 3
3.2 Flowchart . . . 4

4 Penjelasan Program Rust **5**
4.1 Struktur Program . . . 5
4.2 Import Library . . . 6
4.3 Struct Sensor . . . 6
4.4 Struct Controller . . . 6
4.5 Struct MonitoringSystem . . . 7

5 Konsep OOP **7**
5.1 Implementasi OOP pada Sensor . . . 7

6 Implementasi Komputasi Numerik **8**
6.1 Kalibrasi Sensor Linear . . . 8
6.2 Sliding Moving Average W3 . . . 8
6.3 Kontrol Pompa . . . 9

7 Hasil Pengujian **9**
7.1 Skenario Pengujian . . . 9
7.2 Perhitungan Manual . . . 9
7.3 Pengujian Validasi Input . . . 10
7.4 Screenshot Hasil Program . . . 10

8 Analisis Hasil **12**
8.1 Analisis Kalibrasi . . . 12
8.2 Analisis Moving Average . . . 12
8.3 Analisis Kontrol Pompa . . . 12
8.4 Analisis Histori . . . 12

9 Kesimpulan **12**

Referensi **13**

1 Pendahuluan

1.1 Latar Belakang

Sistem monitoring level air merupakan salah satu aplikasi penting dalam bidang instrumentasi. Tandon air pada gedung memerlukan pemantauan secara berkala agar ketersediaan air tetap terjaga dan kondisi ekstrem seperti tandon hampir kosong atau tandon terlalu penuh dapat dihindari.

Pada project ini dikembangkan sebuah aplikasi berbasis terminal menggunakan bahasa pemrograman Rust. Program menerima input level air dalam satuan persen, melakukan validasi input, melakukan kalibrasi linear sensor, menghitung *sliding moving average* dengan 3 data terakhir, menentukan status level air, serta mengontrol pompa secara otomatis.

Sistem ini juga menyimpan histori pembacaan sehingga user dapat melihat riwayat data yang telah diproses selama program berjalan. Dengan demikian, program tidak hanya menampilkan kondisi sesaat, tetapi juga menyediakan catatan monitoring yang dapat digunakan untuk analisis sederhana.

1.2 Tujuan

Tujuan dari project ini adalah sebagai berikut:

- Membuat sistem monitoring level air tandon gedung berbasis Rust.
- Menerapkan konsep OOP menggunakan `struct` dan `impl`.
- Mengimplementasikan validasi input sensor.
- Mengimplementasikan kalibrasi linear sensor.
- Mengimplementasikan *sliding moving average* dengan 3 data terakhir.
- Mengembangkan logika kontrol pompa otomatis berdasarkan nilai moving average.
- Menampilkan dashboard terminal dan histori pembacaan.

2 Studi Kasus Instrumentasi

2.1 Deskripsi Sistem

Sistem yang dikembangkan adalah **Sistem Monitoring Level Air Tandon Gedung**. Tandon air gedung dipantau menggunakan sensor ultrasonik yang disimulasikan melalui input dari user. Input level air diberikan dalam satuan persentase, yaitu 0–100%.

Sistem memproses data input melalui beberapa tahap, yaitu validasi input, kalibrasi sensor, filtering menggunakan *sliding moving average*, klasifikasi status level air, dan kontrol pompa otomatis. Program juga menyediakan perintah `h` untuk menampilkan histori pembacaan dan perintah `q` untuk menghentikan program.

2.2 Parameter Sistem

Tabel 1: Parameter Sistem Monitoring

Parameter	**Nilai**	**Keterangan**
Range sensor	0–100%	Persentase level air
Gain kalibrasi	1.02	Faktor pengali kalibrasi
Offset kalibrasi	-1.0	Faktor koreksi kalibrasi
Buffer moving average	3 sampel	Jumlah data terbaru untuk filter
Batas pompa ON	$< 30\%$	Pompa menyala saat level rendah
Batas pompa OFF	$\geq 80\%$	Pompa mati saat level tinggi

2.3 Status Level Air

Status level air ditentukan berdasarkan nilai moving average.

Tabel 2: Klasifikasi Status Level Air

Status	**Range MA**	**Keterangan**
KRITIS	0–9%	Tandon hampir kosong
RENDAH	10–29%	Segera isi tandon
NORMAL	30–79%	Level aman
TINGGI	80–89%	Mendekati penuh
OVERFLOW	$\geq 90\%$	Tandon penuh

2.4 Aturan Kontrol Pompa

Kontrol pompa dilakukan berdasarkan nilai moving average.

Tabel 3: Aturan Kontrol Pompa

Kondisi Moving Average	**Status Pompa**
$\text{MA} < 30\%$	Pompa NYALA
$\text{MA} \geq 80\%$	Pompa MATI
$30\% \leq \text{MA} < 80\%$	Pompa mempertahankan kondisi terakhir

Penggunaan batas bawah dan batas atas bertujuan untuk mengurangi perubahan status pompa yang terlalu sering ketika level air berada di area tengah.

3 Algoritma dan Flowchart

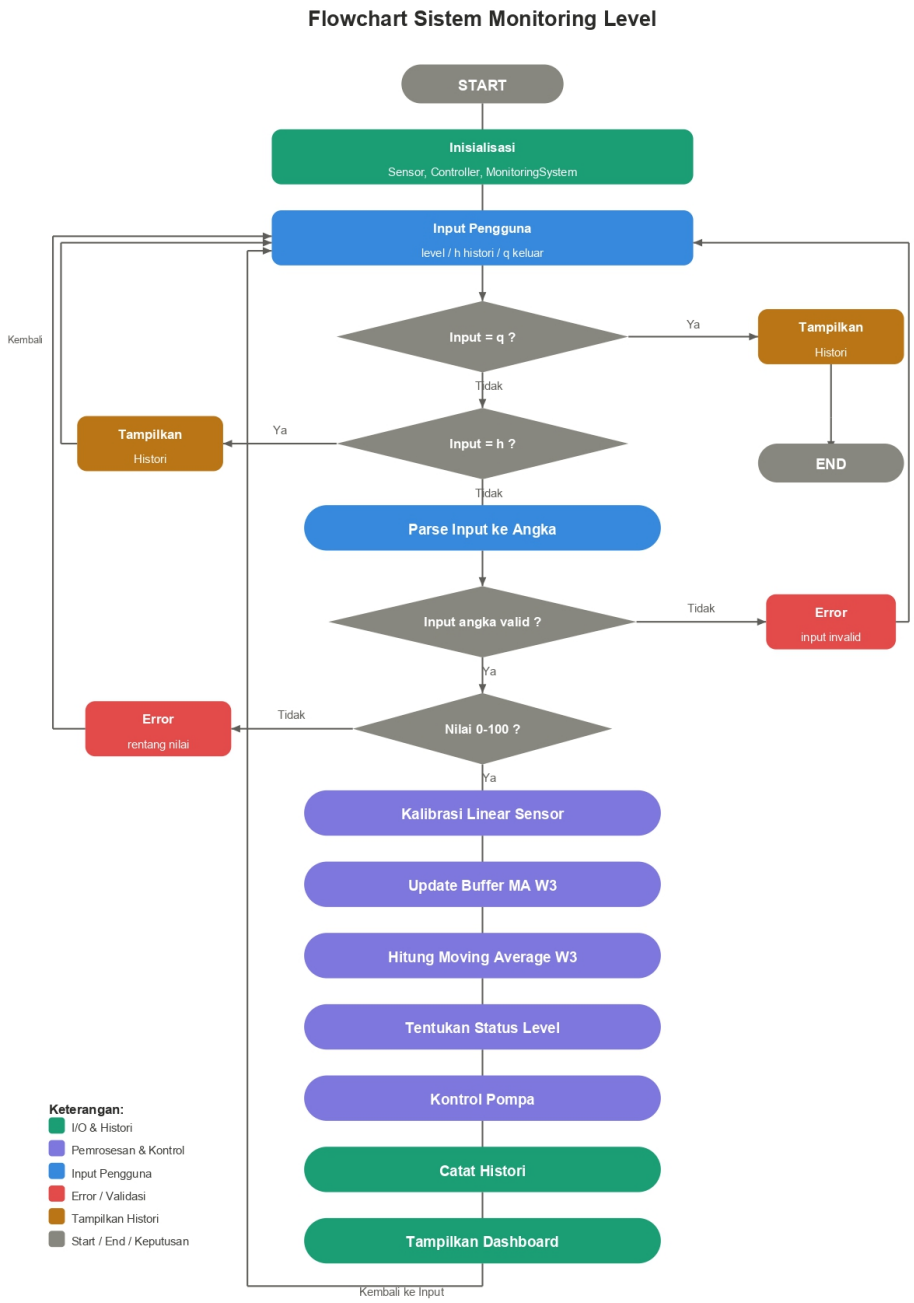
3.1 Algoritma Sistem

Algoritma utama sistem monitoring adalah sebagai berikut:

1. Program dimulai.
2. Sistem menginisialisasi objek `Sensor`, `Controller`, dan `MonitoringSystem`.
3. User memasukkan level air, mengetik `h` untuk melihat histori, atau mengetik `q` untuk keluar.
4. Jika input adalah `q`, sistem menampilkan histori dan program berhenti.
5. Jika input adalah `h`, sistem menampilkan histori dan kembali meminta input.
6. Jika input bukan angka, sistem menampilkan pesan error.
7. Jika input angka berada di luar rentang 0–100, sistem menampilkan pesan error.
8. Jika input valid, sensor menyimpan nilai raw.
9. Sensor melakukan kalibrasi linear menggunakan persamaan $y = (x \times 1.02) - 1.0$.
10. Nilai kalibrasi dimasukkan ke buffer moving average.
11. Jika buffer berisi lebih dari 3 data, data paling lama dihapus.
12. Sistem menghitung moving average dari maksimal 3 data terakhir.
13. Sistem menentukan status level air berdasarkan nilai moving average.
14. Controller mengatur pompa berdasarkan batas bawah dan batas atas.
15. Sistem mencatat data ke histori.
16. Sistem menampilkan dashboard monitoring.
17. Program kembali ke input pengguna.

3.2 Flowchart

Flowchart sistem monitoring level air ditunjukkan pada Gambar 1.



Gambar 1: Flowchart Sistem Monitoring Level Air Tandon

4 Penjelasan Program Rust

4.1 Struktur Program

Program terdiri dari satu file utama, yaitu `main.rs`. Program menggunakan tiga struct utama, yaitu `Sensor`, `Controller`, dan `MonitoringSystem`.

4.2 Import Library

```rust
use std::io::{self, Write};
```

Listing 1: Import library

Program hanya menggunakan Rust Standard Library. Modul `io` digunakan untuk membaca input keyboard, sedangkan `Write` digunakan untuk melakukan *flush* output terminal setelah perintah input ditampilkan.

4.3 Struct Sensor

```rust
struct Sensor {
    nama: String,
    nilai_raw: f32,
    nilai_kal: f32,
    error: bool,
    buffer: Vec<f32>,
}
```

Listing 2: Definisi Struct Sensor

Struct `Sensor` menyimpan data sensor ultrasonik yang disimulasikan melalui input user:

- `nama`: nama sensor.
- `nilai_raw`: nilai input sebelum kalibrasi.
- `nilai_kal`: nilai setelah kalibrasi.
- `error`: status validasi input.
- `buffer`: penyimpanan maksimal 3 data terbaru untuk moving average.

4.4 Struct Controller

```rust
struct Controller {
    batas_bawah: f32,
    batas_atas: f32,
    pompa: bool,
}
```

Listing 3: Definisi Struct Controller

Struct `Controller` digunakan untuk mengatur pompa berdasarkan nilai moving average. Pompa menyala ketika moving average kurang dari 30%, dan pompa mati ketika moving average mencapai 80% atau lebih.

4.5 Struct MonitoringSystem

```
struct MonitoringSystem {
    siklus: u32,
    histori: Vec<String>,
}
```

Listing 4: Definisi Struct MonitoringSystem

Struct `MonitoringSystem` digunakan untuk menghitung jumlah siklus pembacaan valid dan menyimpan histori pembacaan sensor.

5 Konsep OOP

Rust tidak menggunakan *class* seperti bahasa pemrograman berorientasi objek lainnya. Namun, konsep OOP dapat diterapkan melalui `struct` dan `impl`. `struct` digunakan untuk menyimpan data, sedangkan `impl` digunakan untuk mendefinisikan method atau perilaku dari struct.

5.1 Implementasi OOP pada Sensor

```
impl Sensor {
    fn baru(nama: &str) -> Self {
        Sensor {
            nama: nama.to_string(),
            nilai_raw: 0.0,
            nilai_kal: 0.0,
            error: false,
            buffer: Vec::new(),
        }
    }

    fn set_nilai(&mut self, v: f32) {
        if v < 0.0 || v > 100.0 {
            self.error = true;
            return;
        }

        self.error = false;
        self.nilai_raw = v;
        self.nilai_kal = ((v * 1.02) + (-1.0)).clamp(0.0, 100.0);
    }
}
```

Listing 5: Method pada Sensor

Method `baru()` berfungsi sebagai konstruktor untuk membuat objek sensor baru. Method `set_nilai()` digunakan untuk menerima input, melakukan validasi, dan melakukan kalibrasi.

6 Implementasi Komputasi Numerik

6.1 Kalibrasi Sensor Linear

Kalibrasi sensor dilakukan menggunakan persamaan linear:

$$y = (x \times \text{gain}) + \text{offset} \tag{1}$$

Pada program ini digunakan:

$$\text{gain} = 1.02 \tag{2}$$

$$\text{offset} = -1.0 \tag{3}$$

Sehingga rumus kalibrasi menjadi:

$$y = (x \times 1.02) - 1.0 \tag{4}$$

Implementasi pada kode adalah sebagai berikut:

```
self.nilai_kal = ((v * 1.02) + (-1.0)).clamp(0.0, 100.0);
```

Listing 6: Implementasi kalibrasi sensor

Fungsi `clamp(0.0, 100.0)` digunakan agar hasil kalibrasi tetap berada pada rentang 0–100%.

6.2 Sliding Moving Average W3

Moving average digunakan untuk mengurangi fluktuasi pembacaan sensor. Program menggunakan *sliding moving average* dengan window 3 data terakhir.

Rumus moving average adalah:

$$MA = \frac{x_1 + x_2 + x_3}{3} \tag{5}$$

Jika data yang tersedia belum mencapai 3, maka moving average dihitung dari seluruh data yang tersedia.

Implementasi pada kode adalah sebagai berikut:

```
fn moving_average(&mut self) -> f32 {
    self.buffer.push(self.nilai_kal);

    if self.buffer.len() > 3 {
        self.buffer.remove(0);
    }

    self.buffer.iter().sum::<f32>() / self.buffer.len() as f32
}
```

Listing 7: Implementasi sliding moving average

6.3 Kontrol Pompa

Logika kontrol pompa menggunakan dua batas, yaitu batas bawah dan batas atas:

$$\text{Pompa} = \begin{cases} \text{NYALA}, & \text{jika } MA < 30\% \\ \text{MATI}, & \text{jika } MA \geq 80\% \\ \text{Tetap}, & \text{jika } 30\% \leq MA < 80\% \end{cases} \tag{6}$$

Implementasi pada kode adalah sebagai berikut:

```
fn update(&mut self, ma: f32) {
    if ma < self.batas_bawah {
        self.pompa = true;
    } else if ma >= self.batas_atas {
        self.pompa = false;
    }
}
```

Listing 8: Implementasi kontrol pompa

7 Hasil Pengujian

7.1 Skenario Pengujian

Pengujian dilakukan dengan memasukkan beberapa nilai level air untuk melihat proses validasi, kalibrasi, moving average, status level air, dan kontrol pompa.

Contoh input pengujian:

```
10
50
90
98
h
q
```

7.2 Perhitungan Manual

Dengan rumus kalibrasi:

$$y = (x \times 1.02) - 1.0$$

didapatkan:

Tabel 4: Perhitungan Kalibrasi dan Moving Average

Siklus	Raw (%)	Kalibrasi (%)	Buffer MA
1	10	9.20	9.20
2	50	50.00	9.20, 50.00
3	90	90.80	9.20, 50.00, 90.80
4	98	98.96	50.00, 90.80, 98.96

Pada siklus keempat, moving average dihitung dari 3 data terakhir:

$$MA = \frac{50.00 + 90.80 + 98.96}{3} = 79.92\% \tag{7}$$

Dengan nilai MA 79.92%, status level air adalah NORMAL dan pompa masih mempertahankan kondisi sebelumnya karena MA berada pada rentang 30–79%.

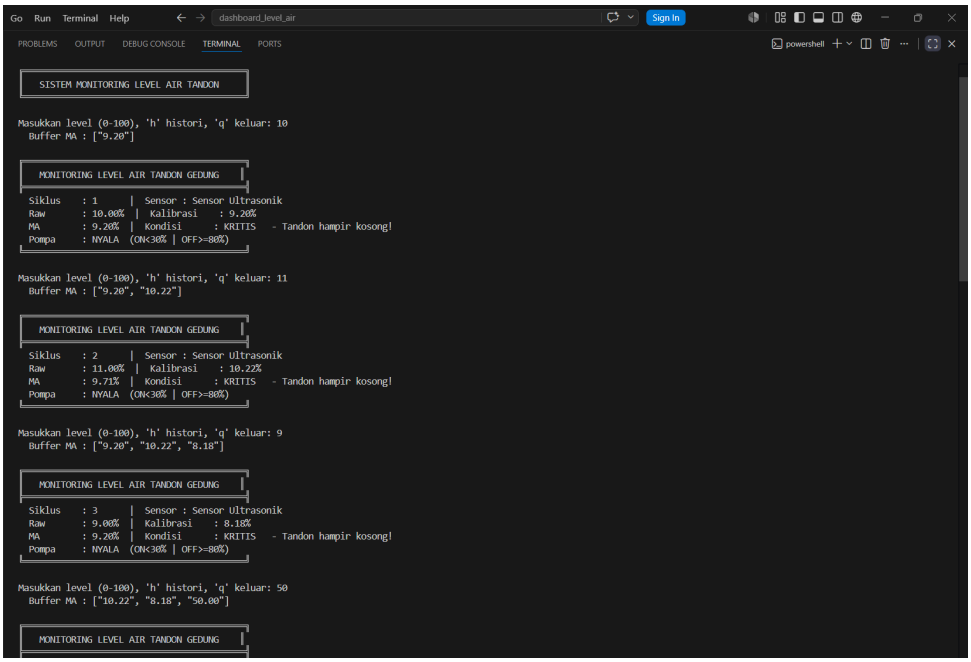
7.3 Pengujian Validasi Input

Jika user memasukkan nilai di luar rentang 0–100, program akan menampilkan pesan error dan data tidak masuk ke perhitungan.

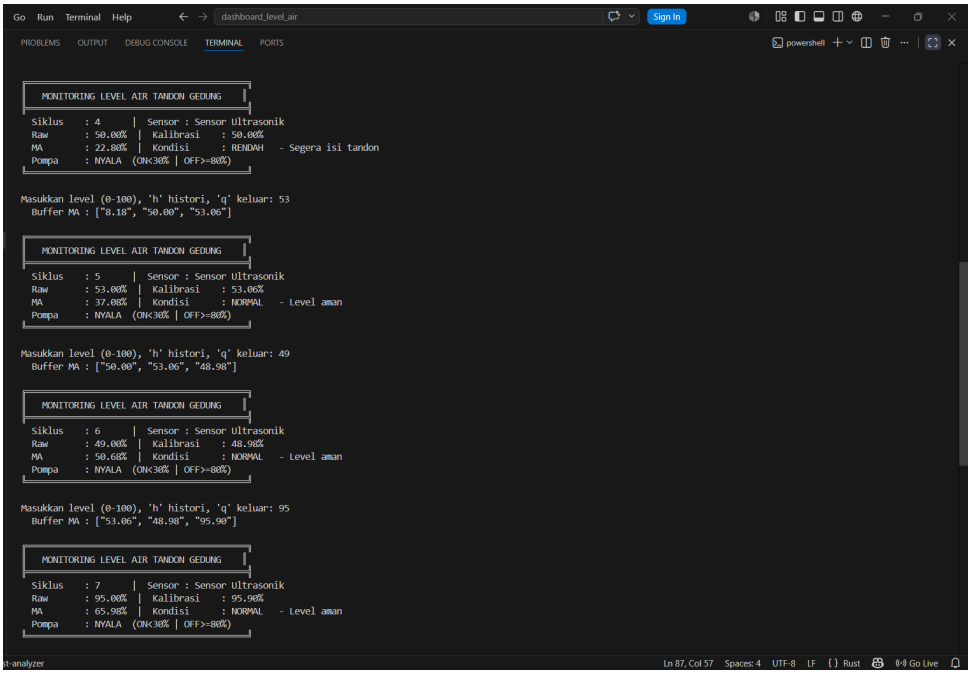
```
Masukkan level (0-100), 'h' histori, 'q' keluar: 120
ERROR: Nilai harus antara 0 sampai 100!
```

7.4 Screenshot Hasil Program

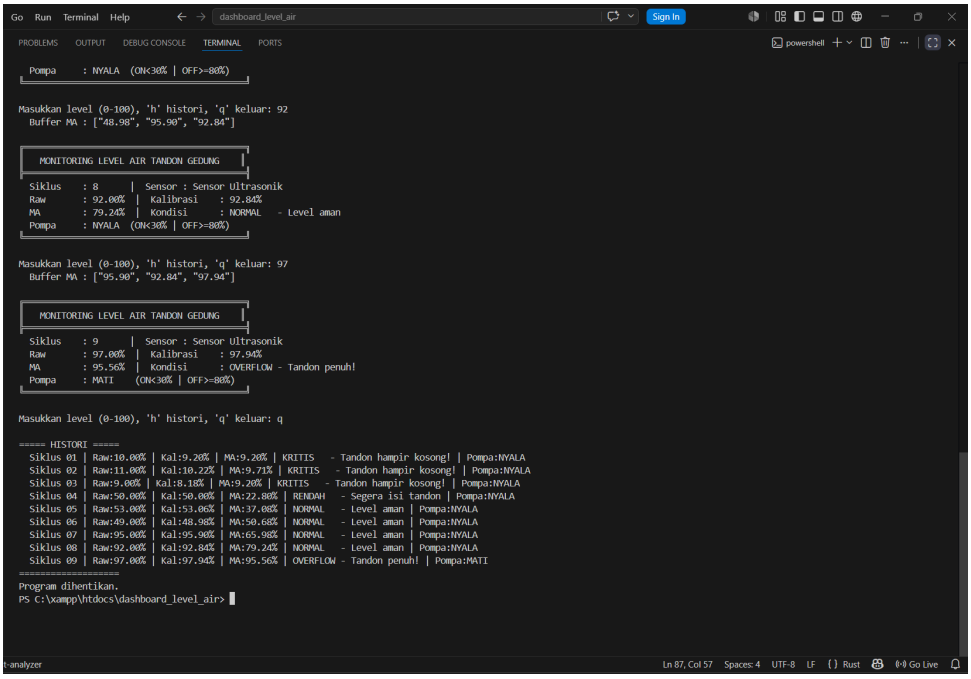
Screenshot hasil program ditunjukkan pada Gambar 2, Gambar 4 dan **??**



Gambar 2: Screenshot Hasil Program 1



Gambar 3: Screenshot Hasil Program 2



Gambar 4: Screenshot Hasil Program 3

8 Analisis Hasil

8.1 Analisis Kalibrasi

Kalibrasi linear berhasil mengubah nilai raw menjadi nilai kalibrasi. Contohnya, input 50% menghasilkan:

$$(50 \times 1.02) - 1.0 = 50.00\%$$

Fungsi `clamp` memastikan hasil kalibrasi tetap berada dalam rentang 0–100%.

8.2 Analisis Moving Average

Moving average W3 berhasil menghaluskan perubahan data sensor. Ketika input berubah dari 50% ke 90% dan 98%, nilai moving average tidak langsung berubah ekstrem, tetapi berubah berdasarkan rata-rata tiga data terbaru. Hal ini membuat sistem lebih stabil terhadap noise atau fluktuasi sesaat.

8.3 Analisis Kontrol Pompa

Kontrol pompa bekerja berdasarkan nilai moving average. Pompa menyala saat MA kurang dari 30% dan mati saat MA mencapai 80% atau lebih. Ketika MA berada di antara 30% dan 80%, pompa mempertahankan kondisi sebelumnya. Logika ini membantu mengurangi perubahan status pompa yang terlalu sering.

8.4 Analisis Histori

Fitur histori memudahkan user untuk melihat seluruh pembacaan valid yang telah dilakukan. Setiap histori mencatat siklus, nilai raw, nilai kalibrasi, moving average, status level air, dan kondisi pompa.

9 Kesimpulan

Berdasarkan hasil perancangan dan pengujian, dapat disimpulkan bahwa:

1. Program berhasil mengimplementasikan sistem monitoring level air tandon gedung berbasis Rust.
2. Konsep OOP berhasil diterapkan menggunakan tiga struct utama, yaitu `Sensor`, `Controller`, dan `MonitoringSystem`.
3. Validasi input sensor berhasil menolak data di luar rentang 0–100%.
4. Kalibrasi linear berhasil diterapkan menggunakan rumus $(x \times 1.02) - 1.0$.
5. Sliding moving average W3 berhasil digunakan untuk menstabilkan pembacaan sensor.

6. Kontrol pompa otomatis berhasil diterapkan berdasarkan nilai moving average.
7. Histori pembacaan berhasil mencatat data monitoring selama program berjalan.

Referensi

1. The Rust Programming Language. https://doc.rust-lang.org/book/
2. Rust Standard Library Documentation. https://doc.rust-lang.org/std/
3. Bolton, W. (2015). *Instrumentation and Control Systems*. Newnes.
4. Modul Algoritma dan Pemrograman, Teknik Instrumentasi ITS, 2026.